

DOCUMENTAZIONE PROGETTO INGEGNERIA INFORMATICA

Smart Navigation App

Sviluppato da: Antonio Falco, Vincenzo Lo Monaco, Luca Seveso.

Durata del Progetto: Febbraio 2026 – Maggio 2026

Docente di riferimento: Matera Mariastella.

1. Introduzione

L'evoluzione vertiginosa dei dispositivi mobili e delle reti di telecomunicazione ha trasformato radicalmente il modo in cui le persone interagiscono con l'ambiente circostante e, in particolare, il modo in cui navigano negli spazi urbani ed extraurbani. I sistemi di navigazione basati su GPS (Global Positioning System), un tempo prerogativa di dispositivi hardware dedicati, sono oggi integrati in ogni smartphone, rendendo la localizzazione geografica un servizio onnipresente e fondamentale. Questa presenza diffusa ha generato un'enorme aspettativa da parte degli utenti in termini di precisione, reattività e usabilità delle interfacce. Tuttavia, nonostante il mercato sia dominato da colossi tecnologici che offrono soluzioni gratuite ed estremamente ricche di funzionalità, l'esperienza utente in scenari specifici presenta ancora significativi margini di miglioramento.

La motivazione primaria che ha guidato la concezione e lo sviluppo di "Smart Navigation App", **un'applicazione dedicata specificamente alle persone con disabilità cognitive**, è stata la volontà di affrontare alcune criticità intrinseche ai navigatori moderni. Molte delle applicazioni più diffuse tendono a soffrire di un fenomeno noto come "feature creep", ovvero l'aggiunta continua e sproporzionata di funzionalità secondarie (social networking, recensioni di ristoranti, pubblicità mirate, suggerimenti musicali) che finiscono per appesantire l'interfaccia grafica e, cosa più critica, aumentare pericolosamente il carico cognitivo dell'utente, creando barriere spesso insormontabili proprio per chi ha necessità di un'esperienza chiara e accessibile.

Un'altra motivazione tecnica fondamentale risiede nell'efficienza computazionale del ricalcolo del percorso. Molti sistemi commerciali utilizzano logiche di "snapping" basate su geofence circolari ampi o attendono l'interrogazione prolungata dei server centrali per confermare una deviazione. Questo può causare frustrazione nell'utente che, avendo sbagliato una svolta, deve percorrere centinaia di metri prima che il sistema proponga una nuova rotta. "Smart Navigation App" si propone di risolvere questo problema alla radice, introducendo algoritmi di deviazione geometrica calcolati direttamente a bordo del dispositivo (edge computing) che garantiscono un ricalcolo praticamente istantaneo.

Obiettivi del Progetto

Il progetto si pone una serie di obiettivi ingegneristici, architetturali e di design ben definiti, progettati specificamente per rispondere alle esigenze delle persone con disabilità cognitive, che hanno guidato l'intero ciclo di vita dello sviluppo software:

- **Sviluppo Cross-Platform Nativo:** Realizzare un'applicazione che possa essere eseguita con prestazioni ottimali (60 frame per secondo) sia su dispositivi Android che iOS, mantenendo una base di codice singola per ridurre i tempi di manutenzione e i costi di sviluppo.

- **Ricalcolo Geometrico Ultra-Rapido (Off-Route Detection):** Implementare un algoritmo matematico proprietario che calcoli

costantemente la distanza ortogonale tra la coordinata GPS in tempo reale e il segmento vettoriale della rotta prevista, minimizzando i tempi di reazione del software in caso di errore dell'utente.

· **Consapevolezza Contestuale Intelligente (Lateral Road Detection):**

Introdurre una funzionalità sperimentale in grado di analizzare le strade secondarie e le intersezioni adiacenti all'utente. Per evitare di distrarre l'utente (un aspetto cruciale per prevenire il disorientamento nelle persone con disabilità cognitive) o saturare le API di terze parti, questa funzionalità deve essere vincolata a rigidi meccanismi di "cooldown" logico che si attivano esclusivamente in condizioni di sicurezza.

· **Architettura a Singola Schermata (Single-Screen Architecture):**

Progettare una gestione dello stato dell'interfaccia utente (UI State Machine) capace di simulare transizioni di pagina fluide all'interno di un'unica vista principale, garantendo la continuità visiva e la prevedibilità necessarie per chi ha fragilità cognitive, ed eliminando i costosi processi di distruzione e ricreazione del motore di rendering della mappa.

Contenuti del Documento

Questo documento tecnico, redatto per la presentazione e la valutazione accademica, fornisce un resoconto esaustivo e dettagliato di tutte le fasi del progetto. La trattazione è organizzata come segue: nella Sezione 2 viene presentata una revisione della letteratura e dei lavori correlati (Related Work), con un'analisi comparativa delle soluzioni attualmente sul mercato; la Sezione 3 evidenzia in modo conciso le reali innovazioni apportate da questo progetto; la Sezione 4, la più corposa, illustra nei minimi dettagli lo sviluppo del progetto, coprendo gli strumenti software, l'architettura dei componenti, le scelte tecnologiche e i pattern di progettazione adottati; la Sezione 5 è dedicata ai Documenti di Specifica, essenziali per la modellazione formale del dominio applicativo; la Sezione 6 presenta i risultati ottenuti attraverso la discussione guidata di screenshot dell'applicativo; infine, la Sezione 7 sintetizza le conclusioni tratte e prospetta gli sviluppi futuri.

2. Related Work (Lavori Correlati)

La navigazione assistita tramite sensori GPS è un campo di studio ampiamente dibattuto nella letteratura accademica e scientifica, oltre ad essere uno dei settori commerciali più competitivi nel panorama mobile. Le ricerche condotte in piattaforme come Google Scholar dimostrano come l'evoluzione storica dei navigatori si sia spostata dalla semplice ottimizzazione degli algoritmi di ricerca su grafo (come Dijkstra) verso l'analisi dei dati in tempo reale (traffico, incidenti) e, più recentemente, verso l'interazione uomo-macchina adattiva (HCI - Human-Computer Interaction).

Gli studi sull'HCI sottolineano l'importanza vitale di ridurre l'inquinamento visivo dell'interfaccia. Nonostante queste evidenze accademiche, le attuali implementazioni commerciali spesso sacrificano la semplicità in favore della monetizzazione (pubblicità sulla mappa) o dell'engagement sociale.

Vista Comparativa delle Soluzioni di Mercato

Un'analisi comparativa dei principali "player" del mercato è fondamentale per comprendere le ragioni che hanno spinto alla creazione di questa applicazione e per definirne il posizionamento strategico:

- **Google Maps:** Essendo il leader incontrastato, Google Maps offre un set di dati cartografici e punti di interesse (POI) che non ha eguali. Utilizza modelli predittivi di intelligenza artificiale per stimare i tempi di percorrenza. Tuttavia, come evidenziato in precedenza, l'applicazione ha subito un processo di "feature bloat". La mappa è spesso satura di icone, e l'algoritmo di deviazione preferisce mantenere la rotta originale il più a lungo possibile ricalcolando il percorso "interno" piuttosto che intercettare rapidamente la nuova direzione vettoriale dell'utente. Inoltre, non fornisce feedback passivo sulle strade circostanti a meno che l'utente non stia attivamente esplorando la mappa a mano.
- **Waze (acquisito da Google):** Nasce con un paradigma completamente differente, basato sul crowdsourcing. La sua forza risiede nelle segnalazioni istantanee della community. Dal punto di vista della navigazione pura, però, Waze è noto per proporre deviazioni estremamente complesse (es. attraversare quartieri residenziali trafficati) per far risparmiare quantità di tempo marginali. L'interfaccia grafica è ricca di animazioni, popup e "caramelle visive".
- **Apple Maps:** Ha compiuto notevoli passi avanti dalla sua travagliata introduzione. Si distingue per un design pulito, minimalista e un rendering vettoriale 3D di altissima qualità, allineandosi meglio ai principi di riduzione del carico cognitivo. Tuttavia, è strettamente confinato all'ecosistema iOS e le sue API per sviluppatori terzi offrono un grado di personalizzazione inferiore rispetto alle controparti.
- **Il nostro Progetto (Smart Navigation App):** Questa applicazione si colloca in una nicchia ben specifica: la navigazione essenziale, proattiva e performante. Rifiutando l'integrazione forzata di recensioni o social network, si ispira al minimalismo di Apple Maps ma lo estende ad Android grazie al framework Flutter. La vera differenziazione risiede nei calcoli a bordo (on-device) per il ricalcolo e nel sistema intelligente di "Lateral Road Detection", che estrae informazioni spaziali (le strade intorno a noi) senza che l'utente debba interagire con lo schermo, rispettando i vincoli di attenzione.

3. Innovazione del Progetto

Nel panorama sovraffollato delle app di mappe, le caratteristiche distintive di questo progetto non risiedono nell'aggiunta di funzionalità collaterali, ma nella re-ingegnerizzazione e nell'ottimizzazione delle logiche "core" della navigazione spaziale.

L'innovazione primaria è rappresentata dal **Lateral Road Detection con Gestione del Cooldown**. I navigatori classici sono "ciechi" rispetto a ciò che non fa parte del percorso calcolato. La nostra applicazione, invece, implementa una scansione passiva del contesto urbano. Tramite le API "Snap-to-Road", quando il veicolo decelera fino a fermarsi (velocità prossima allo zero dedotta dai sensori GPS), l'app identifica la conformazione delle strade laterali all'incrocio in cui ci si trova. La vera novità algoritmica è la gestione temporale di questo processo: per evitare chiamate ridondanti (che consumerebbero banda dati e batteria), il sistema utilizza un timer logico di cooldown e valuta la distanza dal prossimo waypoint. Se ci si trova in una rotonda critica, la funzione viene soppressa per non oscurare la manovra principale. Questa intelligenza contestuale è unica nel suo genere.

La seconda innovazione architetture è il **Modello Single-Screen con Back-Stack Virtuale**. Nello sviluppo mobile tradizionale, passare dalla pagina di "Ricerca" a quella di "Mappa" comporta transizioni pesanti e distruzione degli oggetti in memoria. Abbiamo implementato un controller di stato personalizzato che mantiene vivo il contesto della mappa in background, gestendo gli stati dell'app (Search, Preview, Navigating) tramite overlay scorrevoli e reattivi. Questo produce un'esperienza utente istantanea e permette all'utente di annullare le azioni (es. tramite il pulsante back di Android) senza ricaricamenti, garantendo una fluidità paragonabile alle applicazioni native a basso livello.

4. Sviluppo del Progetto (Project Development)

La realizzazione di "Smart Navigation App" ha richiesto un ciclo di sviluppo iterativo e strutturato, affrontando complesse sfide legate alla gestione dello stato asincrono, all'interfacciamento hardware (sensori GPS) e all'ottimizzazione del rendering grafico. Questa sezione delinea dettagliatamente gli strumenti utilizzati, la piattaforma scelta e l'architettura logico-funzionale del sistema.

4.1. Scelta della Piattaforma e degli Strumenti (Used Tools)

La selezione dello stack tecnologico è stata guidata dai requisiti di cross-compatibilità, prestazioni real-time e disponibilità di librerie geografiche robuste.

Framework di Sviluppo: Flutter

Flutter, sviluppato da Google, è stato scelto come framework di riferimento per l'intero progetto. A differenza di approcci ibridi basati su webview (come Cordova o Ionic) o bridge JavaScript (come React Native), Flutter compila direttamente in codice macchina nativo (ARM o x86) e utilizza il proprio motore di rendering ad alte prestazioni (Skia, e più recentemente Impeller). Questo garantisce che i complessi aggiornamenti visivi della mappa, come la rotazione della telecamera che insegue l'orientamento della direzione da seguire a 60 frame al secondo, avvengano in modo fluido e senza colli di bottiglia (jank). L'architettura dichiarativa basata sui "Widget" ha permesso di comporre interfacce complesse unendo mattoni fondamentali e riutilizzabili.

Linguaggio di Programmazione: Dart

Il linguaggio Dart è profondamente integrato con Flutter ed è stato essenziale per la riuscita del progetto. La sua sintassi moderna ha prevenuto in fase di compilazione innumerevoli bug legati ad eccezioni di riferimenti nulli (NullPointerException), tipici della gestione di dati JSON esterni non sempre prevedibili. Dart brilla particolarmente nella gestione dell'asincronia tramite i concetti di Future (per le singole chiamate API) e Stream (per flussi continui di dati come le coordinate GPS). L'Event Loop di Dart ha permesso di elaborare la logica di calcolo del percorso senza bloccare l'interfaccia utente.

Google Maps Platform e API

Per garantire la massima precisione cartografica, il progetto si appoggia alle API enterprise di Google, implementate attraverso servizi intermediari (Proxy Services) all'interno dell'app:

- **Maps SDK:** Integrato tramite il pacchetto ``google_maps_flutter``, fornisce il layer visivo. Permette il controllo programmatico della telecamera (zoom, tilt, target) e l'inserimento di oggetti geometrici sulla mappa (Polylines e Markers).
- **Directions API:** È il motore algoritmico remoto. Fornendo le coordinate di partenza e arrivo, restituisce un payload JSON contenente il percorso ottimale, le durate stimate (considerando o meno il traffico storicizzato) e una serie di "steps" (istruzioni passo-passo) completi di coordinate di svolta.
- **Places API (Autocomplete):** Cruciale per la fase di ricerca. Consente all'utente di digitare indirizzi incompleti o nomi di attività commerciali, ricevendo suggerimenti in tempo reale e convertendo il testo finale in precise coordinate geografiche (Geocoding indiretto).
- **Roads API (Snap to Road):** Utilizzata in modo innovativo per la feature di Lateral Road Detection. Date alcune coordinate grezze e potenzialmente imprecise, questa

API restituisce i punti esatti adagiati sull'asse stradale più probabile e i relativi identificativi (Place ID), permettendo di determinare con certezza su quale strada l'utente è fermo.

Librerie Specifiche del Progetto:

Oltre all'SDK standard, sono state fondamentali librerie di terze parti gestite tramite il gestore di pacchetti `pub.dev`:

1. `geolocator`: Ha astratto la complessità dei permessi nativi (Android/iOS) per l'accesso ai servizi di localizzazione. Offre un'interfaccia unificata per ottenere la posizione corrente ad alta precisione e, soprattutto, espone uno `Stream` per ricevere aggiornamenti periodici ottimizzati per il risparmio energetico.
2. `flutter_polyline_points`: Il tracciato restituito da Google Directions API è codificato in una singola stringa alfanumerica complessa (Google Encoded Polyline Algorithm Format). Questa libreria decodifica efficientemente la stringa in un array utilizzabile di oggetti `Lat-Lng`, indispensabile per tracciare la linea blu sulla mappa.

4.2. Progettazione Architetture e Design Pattern

L'applicazione è stata ingegnerizzata seguendo rigorosamente un'architettura a livelli (Layered Architecture) fortemente ispirata ai pattern MVVM (Model-View-ViewModel) e all'Inversion of Control tramite l'utilizzo di Service Locator o Dependency Injection (per gestire i servizi singleton).

L'obiettivo architettonico principale era disaccoppiare la logica spaziale complessa (calcoli matematici sulle coordinate) dal livello di presentazione visiva (bottoni e mappe). Questo è stato ottenuto strutturando il codice in domini chiari:

Livello dei Modelli Dati (Data Models)

Ogni risposta grezza in formato JSON proveniente dai server remoti viene immediatamente tradotta (parsing) in oggetti di dominio fortemente tipizzati, garantendo coerenza in tutta l'applicazione. Esempi chiave sono la classe `RouteData` (che incapsula l'intero viaggio, polilinee complessive, bound rettangolari), la classe `Step` (che rappresenta una singola istruzione come "Svolta a destra tra 100 metri") e `NavigationSession` (un modello che memorizza le statistiche del viaggio corrente, distanza percorsa e tempo rimanente).

Livello dei Servizi di Rete (Network Services)

Classi come `DirectionsService` e `PlacesService` agiscono come astrazioni di rete. Il loro unico compito è formattare l'URL corretto, iniettare le chiavi API in modo sicuro (tramite file `.env` escluso dal repository di versionamento) e eseguire la chiamata asincrona.

Livello di Logica e Orchestrazione (Domain Logic) - Il Cuore del Sistema

Il nucleo intellettuale dell'applicazione risiede in un servizio orchestratore rotocollo HTTP/HTTPS, gestire gli errori di connessione (timeout, assenza di rete) e restituire al livello superiore un oggetto pulito o sollevare un'eccezione tipizzata e gestibile denominato `NavigationMonitor`. Questa classe non sa nulla dell'interfaccia utente visiva, ma mantiene l'intero stato logico della navigazione in corso. Le sue responsabilità sono:

- Sottoscrizione allo stream del `geolocator`.
- Calcolo della Cross-Track Error (XTE): ad ogni battito del GPS, calcola la distanza ortogonale dalla polyline. Se questa supera una soglia predefinita (es. 25 metri),

innesca la complessa procedura asincrona di ricalcolo del percorso interpellando i servizi di rete, aggiornando la polyline e notificando la UI.

- **Gestione dei Waypoint (Steps):** Controlla costantemente la distanza dall'imminente manovra. Al diminuire della distanza, aggiorna la proprietà di stato per istruire l'interfaccia utente ad aggiornare il testo a schermo (es. da "Prosegui dritto" a "Svolta a destra").
- **Rilevamento Arrivo:** Implementa una logica basata sulla prossimità alla coordinata finale per dichiarare conclusa la sessione di navigazione ed emettere eventi celebrativi.

L'elaborazione matematica necessaria al `NavigationMonitor` è astratta all'interno di un modulo `geo_utils.dart`. Lì risiedono implementazioni di trigonometria sferica, come la formula di Haversine per la distanza tra coordinate e calcoli vettoriali per le tangenti necessarie a disegnare le frecce direzionali (chevron) orientate correttamente lungo il tracciato curvo.

Livello di Presentazione e Gestione dello Stato (UI & State Management)

L'interfaccia è composta da un widget genitore, il `MapWidget`, che ospita il `GoogleMap` nativo. A sovrapporsi alla mappa, con sfondi traslucidi (glassmorphism), vi sono una serie di pannelli fluttuanti gestiti dal `NavigationHistoryService`. Questo servizio implementa uno "State Machine" reattivo (tramite paradigmi simili a `'ValueNotifier'` o `'Provider'`). Gli stati dell'UI sono esplicitati in un'enumerazione (Enum): `idle` (ricerca), `preview` (percorso visualizzato ma non iniziato), e `navigating` (guida attiva). Quando l'utente preme il pulsante fisico "Indietro" di Android, il sistema non distrugge l'Activity, ma intercetta l'evento delegandolo al servizio di history, il quale esegue un "pop" dello stato, rimuovendo fluidamente la navigazione attiva e riportando l'interfaccia alla modalità di anteprima, il tutto in pochi millisecondi e senza caricamenti di rete.

4.3. Implementazione di Funzionalità Avanzate

Questa sottosezione approfondisce le sfide incontrate nell'implementazione di algoritmi complessi, andando oltre le configurazioni basilari fornite dai framework commerciali.

1. Generazione Dinamica di Frecce Direzionali (Chevron) sulla Polyline

Una delle richieste cruciali per una UX completa era l'indicazione visiva chiara della direzione di marcia. L'SDK standard di Google Maps disegna semplicemente linee continue. Per realizzare i chevron orientati, è stato sviluppato un algoritmo iterativo che:

- a. Analizza ogni singolo segmento della polyline del percorso (composta spesso da centinaia di coordinate continue).
- b. Calcola l'inclinazione (bearing) del segmento rispetto al Nord magnetico utilizzando formule trigonometriche.
- c. Genera, a intervalli regolari basati sulla distanza accumulata in pixel/metri, un marker personalizzato utilizzando un asset grafico a forma di freccia.
- d. Ruota matematicamente l'asset grafico per allinearlo all'inclinazione del segmento calcolata al punto (b). Questo garantisce che, anche in percorsi tortuosi come rotonde o tornanti montani, la freccia punti sempre verso la continuazione esatta della strada, aumentando l'intuitività della navigazione.

2. Rilevamento Geometrico della Deviazione (Off-Route Tracking)

La metodologia standard per capire se un utente è "sul percorso" prevede di tracciare un cerchio (geofence) attorno alla posizione corrente e verificare se la coordinata di

destinazione o il prossimo snodo vi è all'interno. Il nostro progetto implementa invece un calcolo di **Distanza Punto-Poligono**. L'algoritmo (incluso in ``geo_utils.dart``) converte la curva terrestre in coordinate piane a livello locale e iterativamente proietta ortogonalmente la posizione attuale dell'utente su ciascun segmento adiacente della traccia principale. Identificato il segmento più vicino, estrae la distanza esatta in metri. Se questa distanza supera una tolleranza empiricamente calcolata (es. 30 metri) per più di tre rilevamenti GPS consecutivi (per scartare errori momentanei del sensore), il sistema innesca il ricalcolo. Questo previene ricalcoli nevrotici in caso di segnale GPS disturbato (urban canyon), garantendo stabilità logica.

3. Ottimizzazione delle Prestazioni tramite Lazy Loading

Per garantire che l'avvio dell'applicazione avvenga in meno di 2 secondi, l'architettura sfrutta ampiamente l'inizializzazione ritardata dei servizi (Lazy Initialization). Solo i servizi critici per il rendering, come la gestione dei permessi, vengono istanziati al lancio; moduli complessi, come il parser di polyline ad alta densità, vengono caricati in memoria asincronicamente solo quando l'utente richiede l'anteprima del percorso. Le icone custom dei marker (come l'icona del veicolo o le frecce direzionali) vengono precompilate (pre-rasterizzate) in array di byte prima del render, riducendo a zero il ritardo di disegno che si verificherebbe se la mappa dovesse caricare l'immagine dal disco a ogni frame ridisegnato.

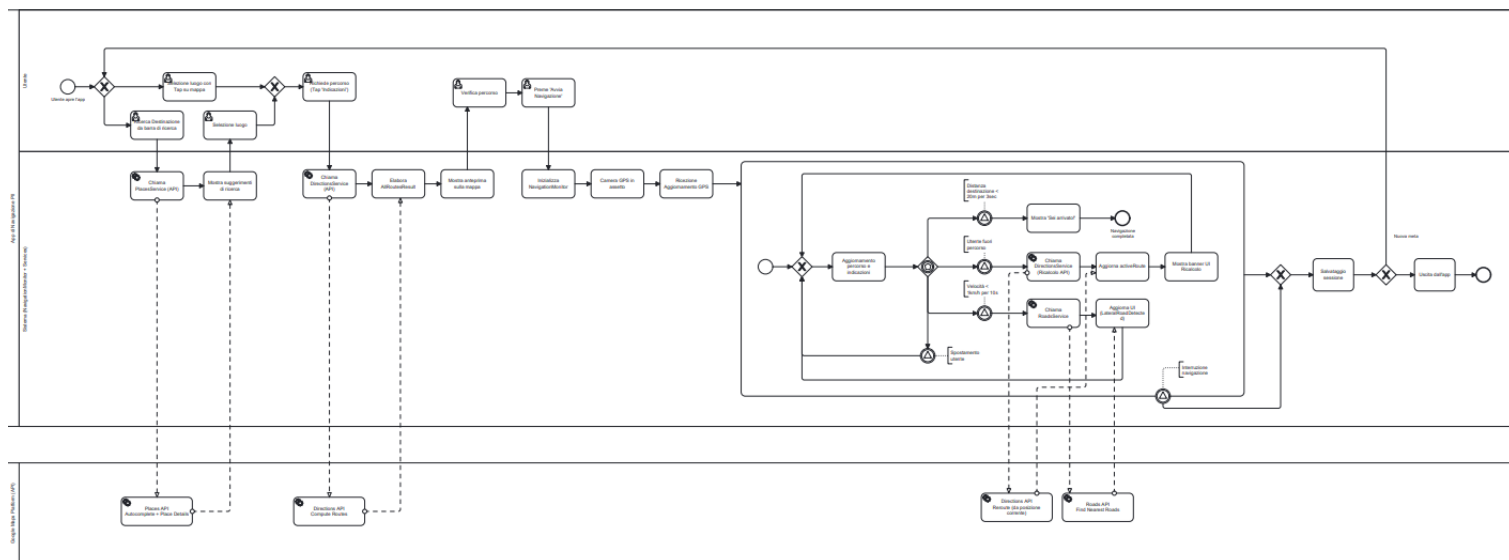
5. Documenti di Specifica (Architecture & Schemas)

Questa sezione è dedicata alla formalizzazione tecnica del sistema attraverso i linguaggi di modellazione standard (UML e BPMN). Sebbene l'astrazione concettuale sia stata implementata in codice, la modellazione rigorosa è stata indispensabile per validare le logiche architetturali prima della stesura del codice, riducendo drasticamente le regressioni funzionali. Di seguito vengono descritti e documentati i principali schemi adottati.

5.1. Processo BPMN (Business Process Model and Notation)

Il diagramma BPMN modella il flusso di alto livello dal punto di vista dell'utente finale interagente con il sistema software, evidenziando i gateway logici (decisioni) e i task eseguiti sia dall'umano che dal computer in modalità asincrona.

Descrizione del Flusso Logico: Il processo inizia con l'evento "Utente apre l'app". All'inizio del processo si ha subito uno XOR con due possibili uscite al fine di cercare una destinazione: "Tap sulla mappa" o "Ricerca testuale". La prima identifica subito il luogo, mentre la seconda chiama la Places API per mostrare i suggerimenti all'utente che deve selezionare un luogo. Una volta scelta la destinazione viene chiamata la DirectionAPI con successiva elaborazione e visualizzazione del percorso. Ora l'utente può avviare la navigazione. Nel corso della navigazione ci sono varie opzioni che possono essere innescate e ognuna di esse genera chiamate o ricalcoli nella view.

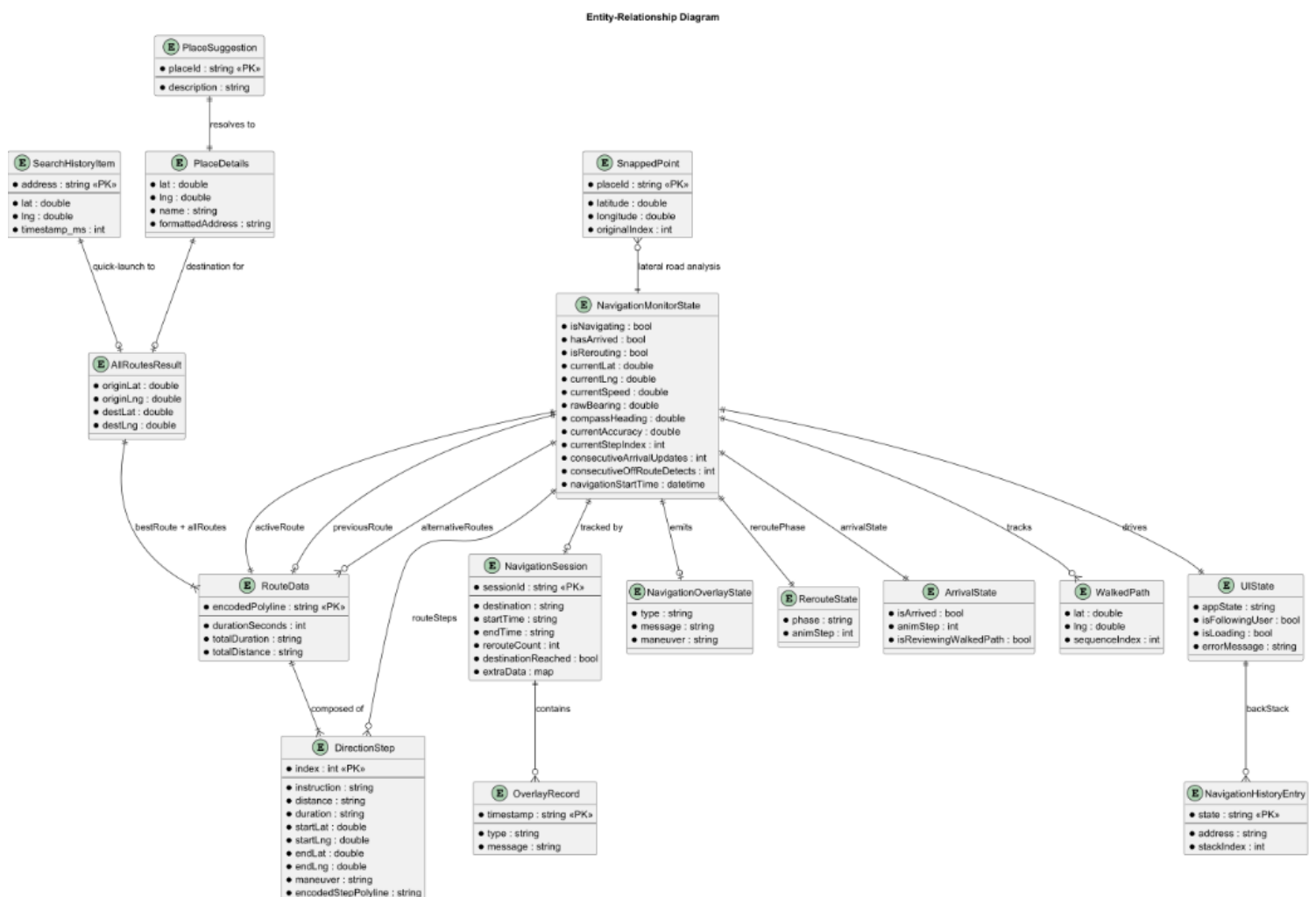


5.2 Schema Entità-Relazione (ER Data Schema) e Struttura

Dati

Nonostante l'applicazione non utilizzi un database relazionale locale pesante (come SQLite) per lo storage persistente a lungo termine dei percorsi complessi, gestisce in RAM una struttura dati articolata che può essere formalizzata attraverso un modello Entità-Relazione.

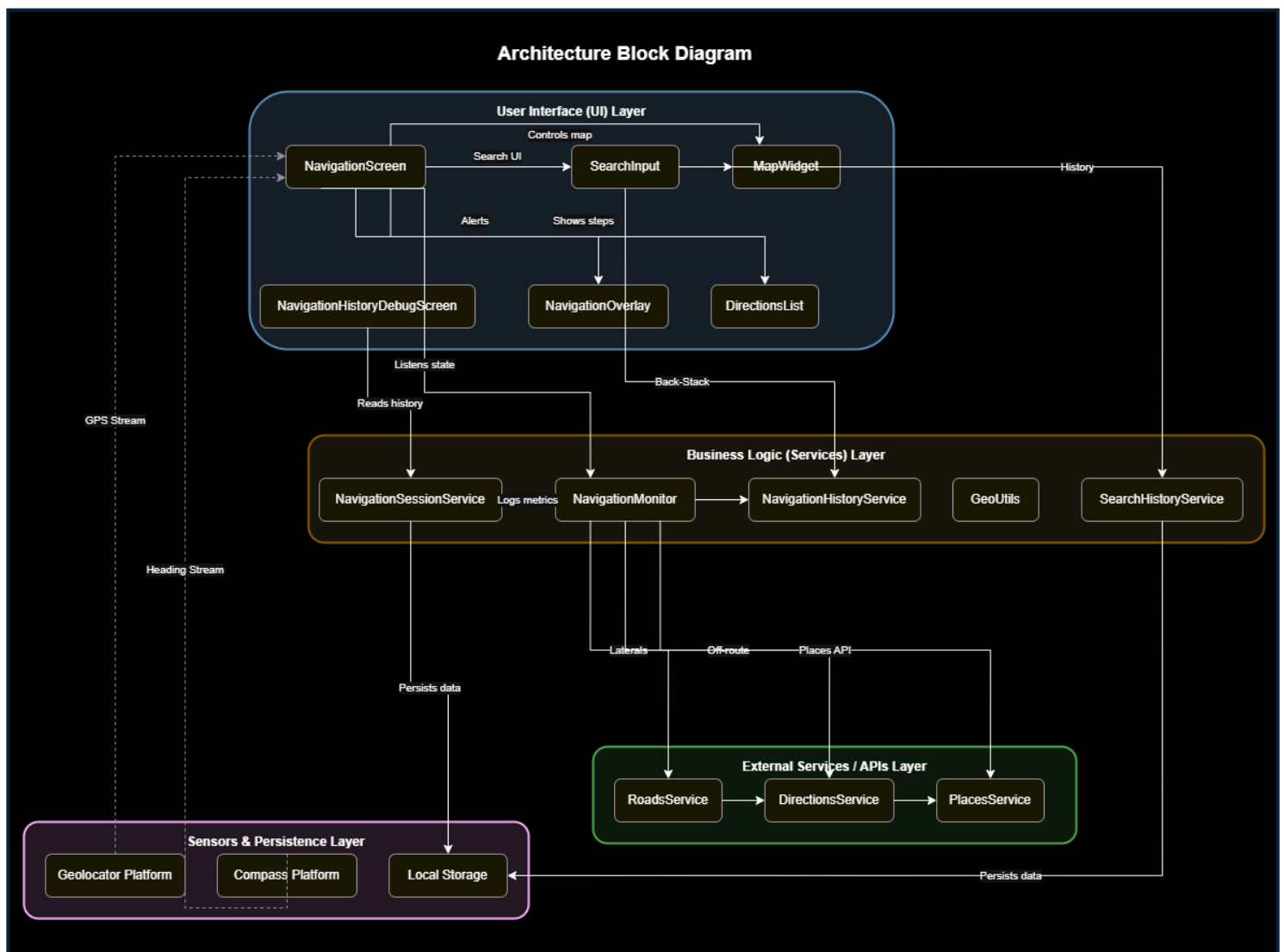
Strutture Dati Principali: Il diagramma ER struttura i dati attorno all'entità di monitoraggio `NavigationMonitorState`, che traccia dinamicamente la telemetria in tempo reale tramite attributi come coordinate e velocità. Il tracciato pianificato è governato dall'entità `RouteData`, contenente attributi essenziali quali la distanza totale (`totalDistance`) e la durata (`durationSeconds`), la quale instaura una relazione uno-a-molti con l'entità subordinata `DirectionStep`. Ogni step definisce il singolo tratto di manovra includendo l'istruzione testuale e le rispettive coordinate esatte di inizio e fine (`startLat`, `endLat`). Un'ulteriore entità gestionale è la `NavigationSession`, che funge da registro per l'intero viaggio salvando i timestamp di avvio (`startTime`) e conclusione (`endTime`), assieme all'esito sull'arrivo a destinazione.



5.3. Diagrammi dei Blocchi (Block Diagram Architeturale)

Il diagramma a blocchi espone i componenti macrostrutturali dell'applicazione e le loro interdipendenze di comunicazione (Component Architecture).

Analisi dei Componenti: Il blocco superiore, **UI Layer**, è suddiviso tra il contenitore della mappa vettoriale e i livelli di overlay (Search Bar fluttuante, Pannelli istruzioni). Questo comunica in maniera unidirezionale (ascolto reattivo) con il blocco intermedio, il **Business Logic Layer**. All'interno di quest'ultimo troviamo i due orchestratori: **NavigationHistoryService** (gestisce la logica dei back-button e i cambi visivi) e **NavigationMonitor** (gestore dei thread asincroni). Questo comunica verso l'esterno. Si divide a sua volta nel blocco "**Sensor & Persistence Layer**" (l'astrazione del modulo GPS hardware tramite 'Geolocator', che fornisce lat/lng, bearing e speed) e il blocco "**External Service**" (le classi wrapper per le chiamate REST verso l'ecosistema Google Cloud). Questa rigida separazione garantisce testabilità: è possibile sostituire virtualmente il blocco "Sensori Dispositivo" iniettando coordinate finte (Mocking) per testare l'algoritmo di deviazione senza dover effettuare un test reale su strada (Drive Test).

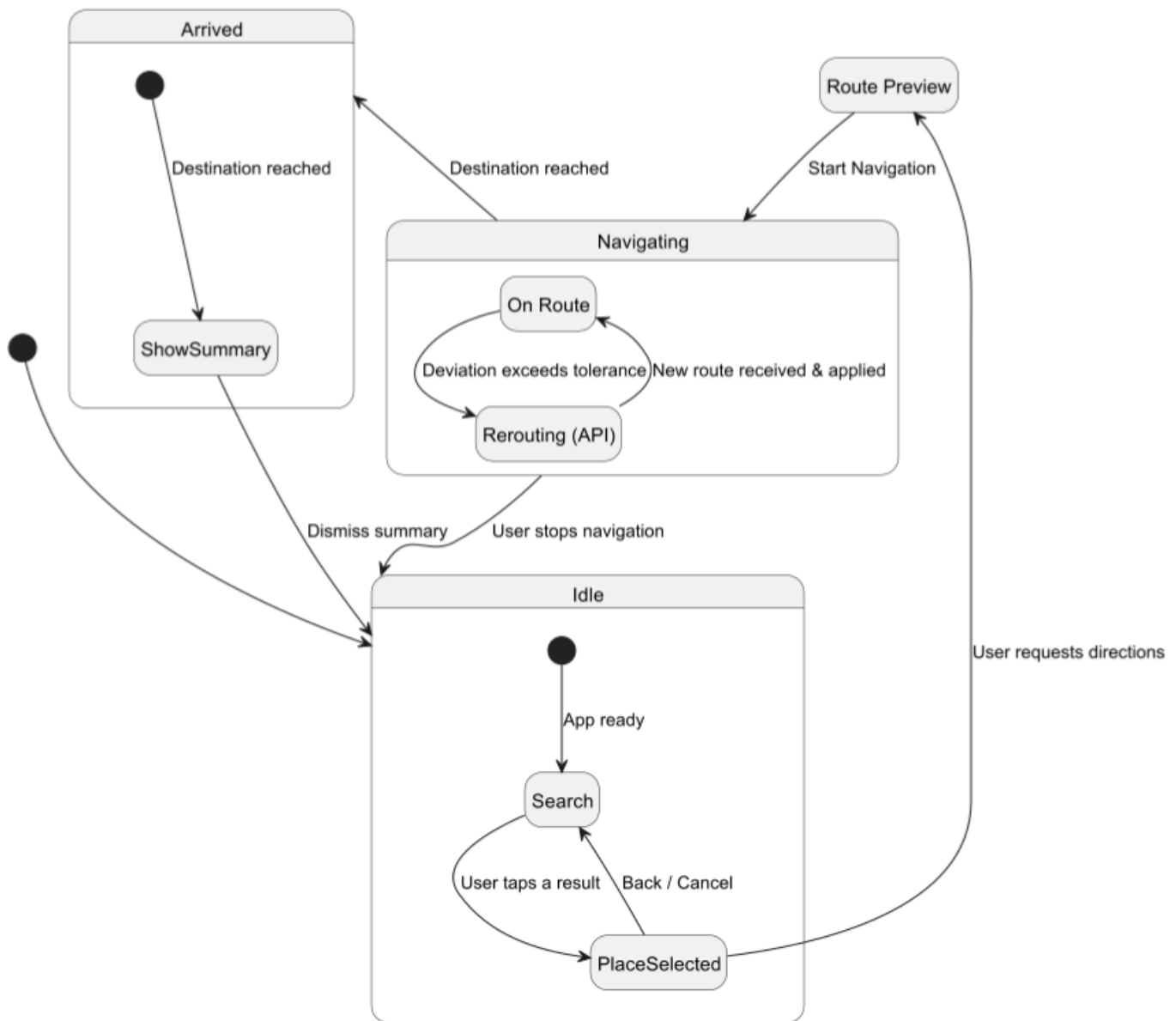


5.4. Diagramma di Stato e Transizioni (State Machine)

Il diagramma di stato illustra il ciclo di vita dell'esperienza di navigazione all'interno dell'applicazione, modellato attraverso quattro macro-stati principali:

1. **Stato Idle (Attesa/Ricerca)** È lo stato iniziale in cui l'applicazione è pronta all'uso. Il flusso si divide in due sottostati:
 - Search: L'utente sta attivamente cercando una destinazione tramite l'interfaccia.
 - PlaceSelected: L'utente ha selezionato un risultato dalla ricerca. Da qui, può decidere di annullare l'operazione (tornando in Search) oppure procedere.
2. **Stato Route Preview (Anteprima Percorso)** Se l'utente, dopo aver selezionato un luogo (PlaceSelected), richiede le indicazioni, il sistema transita in Route Preview. In questa fase viene calcolato il tragitto e mostrato sulla mappa in modalità di sola visualizzazione, permettendo all'utente di esaminare la rotta prima della partenza.
3. **Stato Navigating (Navigazione Attiva)** Inizia quando l'utente conferma l'avvio del viaggio ("Start Navigation" da Route Preview). È il cuore del sistema e gestisce dinamicamente il tracciamento tramite due sottostati:
 - On Route: La posizione GPS dell'utente coincide (entro i margini di tolleranza) con la rotta stabilita.
 - Rerouting (API): Se il sistema rileva una deviazione significativa dal tracciato ("Deviation exceeds tolerance"), entra temporaneamente in questo stato per interrogare i servizi di routing. Una volta ricevuta e applicata la nuova traiettoria, ritorna in On Route. In qualsiasi momento, l'utente può interrompere la navigazione ("User stops navigation"), forzando il sistema a tornare allo stato iniziale Idle.
4. **Stato Arrived (Arrivo a Destinazione)** Quando i dati GPS confermano il raggiungimento delle coordinate finali ("Destination reached"), la navigazione attiva si conclude e si transita nello stato Arrived. Al suo interno si attiva il sottostato ShowSummary, che presenta all'utente un riepilogo visivo o testuale del viaggio appena concluso. Una volta che l'utente chiude ("Dismiss summary") questa schermata, il ciclo si chiude e l'app ritorna allo stato Idle, pronta per una nuova ricerca.

State Machine - Navigation App States



5.5. Diagramma delle Classi UML (Class Diagram)

Il diagramma delle classi offre una vista statica del codice sorgente, illustrando le entità object-oriented scritte in Dart, i loro attributi privati, i metodi pubblici esposti e i pattern di relazione (composizione, aggregazione, ereditarietà).

Focus Architetture sulle Classi: La classe `NavigationMonitor` si configura come Singleton (una sola istanza globale). Possiede attributi come `StreamSubscription _positionStream` per mantenere il riferimento in memoria vitale all'hardware. Espone metodi pubblici reattivi come `ValueNotifier get isOffRouteNotifier`, che l'interfaccia UI ascolta per disegnare avvisi rossi in tempo reale. Si collega per Associazione alla classe `geo_utils`, che raccoglie metodi statici puri (senza stato), i quali accettano input geografici e ritornano risultati matematici, garantendo alta testabilità unitaria. Il livello Servizi mostra la gerarchia delle dipendenze, evidenziando ad esempio come la classe UI visiva `MapWidget` dipenda strettamente dal controller fornitogli dalla libreria `google_maps_flutter`, e non direttamente dalle classi di business logic, le quali le vengono passate tramite iniezione (Dependency Injection), mantenendo l'architettura SOLID (Single Responsibility Principle, Open/Closed Principle).

[illegible]

1. L'utente preme su un luogo.
2. Il Sistema calcola il bounding box che collega l'origine e la destinazione.
3. Il Sistema invia la richiesta asincrona a Directions API.
4. Il Sistema elabora il JSON di risposta tracciando la Polyline ("best route") sulla mappa.
5. Il Sistema espone la vista utente completa di banner della durata in minuti e della distanza in km.

3. Avviare e monitorare la navigazione

Descrizione: Inizia il monitoraggio attivo del GPS lungo il tragitto pianificato. **Attori:** Utente, Sistema. **Flusso Principale:**

1. L'utente preme il pulsante primario "Avvia".
2. Il Sistema acquisisce il lock sul GPS e sul magnetometro iniziando lo stream periodico.
3. Il Sistema incrocia costantemente le coordinate utente con gli Steps calcolati.
4. La telecamera automatizza l'angolo visuale e segue lo spostamento reale via Polyline.
5. Il Sistema dirama eventi visuali alla UI in base alle posizioni (istruzioni di svolta e notifiche push contestuali).
6. Una volta vicinissimo alla meta, viene validata la sequenza ed emesso il BottomSheet "Sei Arrivato".

4. Ricevere analisi strade parallele

Descrizione: Il sistema valuta l'incostanza dello stream e protegge contro errori GPS qualora l'utente sia fermo di fianco ad incroci equivoci. **Attori:** Sistema, Servizi Esterni (Roads API). **Flusso Principale:**

1. Il sensore GPS stima una derivazione di velocità molto vicina allo 0.
2. Un timer di dieci secondi esegue un countdown controllato dal Sistema.
3. Lo stream interpella la Roads API passandogli le coordinate spazzate lateralmente.
4. Constatata un'alta confidenza in strade vicine, il Sistema proietta il rinforzo positivo "continua dritto stai andando bene".

5. Ricalcolo del percorso (Rerouting)

Descrizione: L'utente prende una strada errata e il percorso viene aggiornato per riagganciare la strada giusta. **Attori:** Utente, Sistema, Servizi Esterni (Directions API). **Flusso Principale:**

1. Il Sistema individua una traslazione logica incompatibile con la tolleranza del segmento attivo (>30m errore).
2. Tenta preliminarmente il match all'interno delle Rotte Alternative precaricate in memoria per velocizzare.
3. In caso di fallimento, un overlay asincrono blocca la UI annunciando "Ricalcolo in corso".
4. Viene consumata una nuova query API per stabilire ex-novo il best route dall'origine corrente.
5. Viene esposta all'utente l'opportunità di approvare il roll-forward sul nuovo itinerario o rifiutarlo.

5.7. Casi d'Uso (Use Cases e Scenari UX)

Architettura dell'interfaccia (Material Design Platform)

L'applicazione fa un uso massivo di componenti nativi ed espansi per il framework Flutter, mantenendo rigorosamente il focus sull'accessibilità cognitiva e riducendo al minimo l'eccesso visivo potenziale.

L'interfaccia segue un pattern Single-Page-Application (SPA) adattato per il mobile: un singolo widget manager, il 'NavigationScreen', coordina le transizioni degli strati senza ricorrere ad un app-routing spaghetto. Gestisce le mutazioni logiche direttamente tramutandole in strati di layer (es. search overlay -> route preview dialog).

Sovrapposizione Z-Index (Livelli della View):

1. **Z-Index 0 (Base):** Il Render Surface occupato dal 'MapWidget' (Google Maps Android Rendering View). Include le Polyline tracciate ma nasconde attivamente elementi distraenti come i MapControls nativi.
2. **Z-Index 1 (Overlay Controlli Superiori):** La barra di ricerca ('SearchInput') tranciabile, reattiva in auto-hide al dismiss della keyboard.
3. **Z-Index 2 (Modal & BottomSheets):** Finestre modali comportamentali ('DirectionsList' e riepiloghi di fine corsa) che scorrono a comparsa dal basso in maniera Draggable. Questa meccanica incoraggia lo swiping agevole da una singola mano.

4. Z-Index 3 (Banner di Allerta e Notifica): Il container `NavigationOverlay`, appeso ad altissima priorità alla root visiva, che interviene in determinati tick del lifecycle iniettando testo rafforzante o avvisi di ricalcolo animati da una mascotte.

Design Pattern per Accessibilità Cognitiva:

- **Testo e Colore Semantico Coordinato:** Le colorway rispondono all'infrastruttura di feedback. Segnali informativi usano blu neutrali, segnali di instabilità (come il lateral road fallback) usano arancioni di warning affievoliti per destare l'attenzione senza indurre panico, segnali di goal completato (arrivo) attivano verdi celebrativi.

- **Costruzione Morfologica Semplice:** Completa assenza del gergo tecnico della navigazione (niente calcoli sui bearing gradi, coordinate GPS o log sui fallback). Totale predisposizione di messaggi attivi a cinque parole, assistiti da emoji di parsing visivo.

- **Grace Period Dinamici (Delayed UI):** Elementi come il checkmark di fine tragitto o l'errore del ricalcolo vengono "rallentati" via Future logic a passaggi di 1.5/2 secondi in un'animazione a 3 step progressivi. Questa scelta protegge gli utenti da una cascata di informazioni immediate che stresserebbe il focus cognitivo.

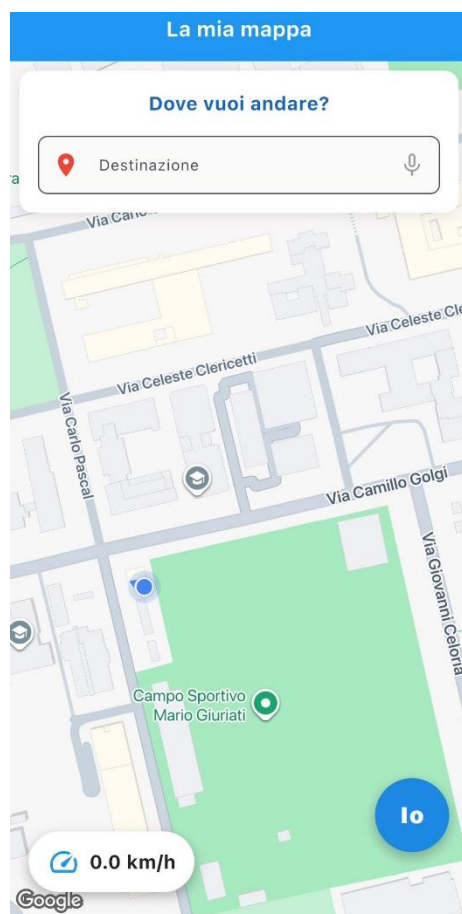
6. Risultati (Results)

La presente sezione illustra l'efficacia concreta dell'architettura e degli algoritmi implementati. Attraverso una rassegna visiva delle funzionalità operative, vengono validati gli obiettivi iniziali del progetto. I test, eseguiti tramite build native su dispositivi fisici mobili, garantiscono l'esattezza dei comportamenti logici descritti nelle specifiche.

6.1. Scenario Operativo: Avvio e Ricerca Immediata

Il primo impatto con l'applicazione definisce l'esperienza utente. Lo screenshot di seguito illustra lo stato iniziale. L'interfaccia si presenta minimalista, dominata dalla mappa vettoriale centrata sulla posizione istantanea dell'utente tramite il marcatore "My Location" di Google Maps. L'assenza di tab bar ingombranti focalizza l'attenzione sulla barra superiore multifunzione.

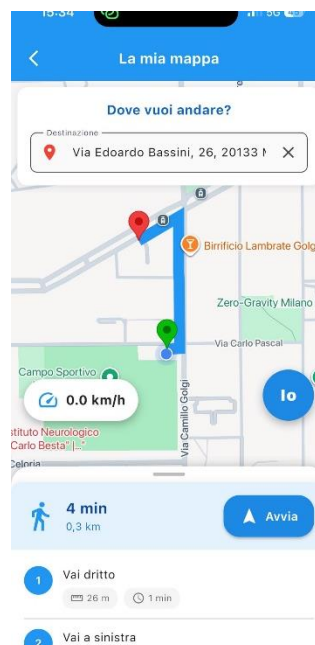
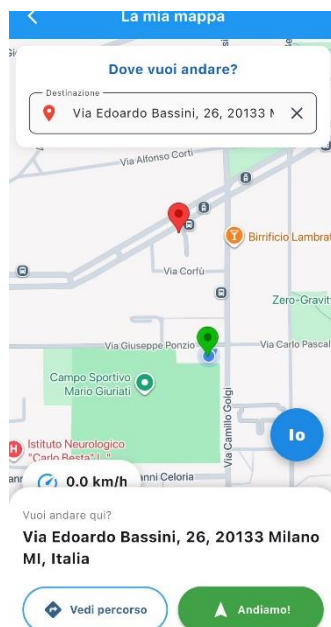
La tastiera si sovrappone in modo fluido e i risultati della chiamata asincrona all'API 'Places Autocomplete' si popolano in tempo reale al di sotto della barra, oscurando parzialmente la mappa per dare risalto al testo tramite un livello di blur o overlay scuro. L'efficienza di questa fase risiede nella decodifica ultrarapida dei JSON in liste di Widget costruite dinamicamente dal motore Dart.



6.2. Scenario Operativo: Generazione dell'Anteprima e Calcolo Tempistiche

A seguito della selezione del luogo di destinazione, l'app calcola istantaneamente il percorso ottimale. La schermata sottostante cattura lo stato di 'PREVIEW'. In questa fase, la telecamera dell'SDK di Google Maps esegue un "CameraUpdate.newLatLngBounds", un algoritmo di zoom che incornicia perfettamente sulla singola visuale del dispositivo sia la posizione di partenza che quella di arrivo (inclusi i marker personalizzati).

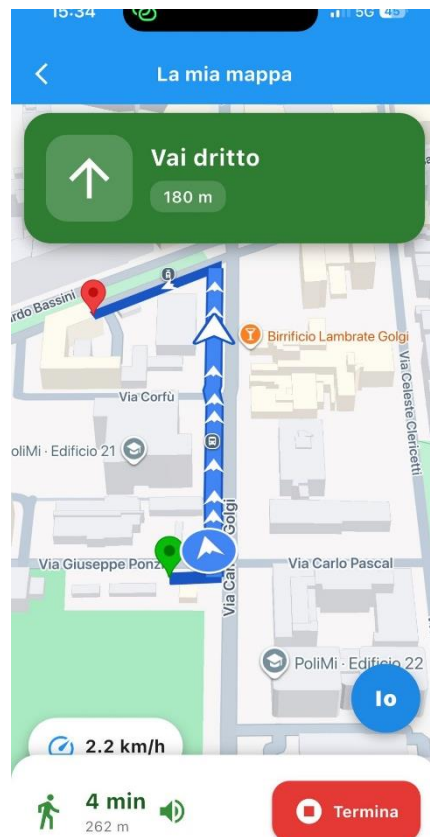
L'elemento visivo predominante è la Polyline blu scura decodificata con successo dall'algoritmo matematico. Nella parte inferiore, emerge un pannello informativo in stile "BottomSheet" persistente. Esso ospita dati vitali estratti dalla risposta dell'API di Directions, formattati in modo leggibile (ad esempio, convertendo i secondi grezzi in ore e minuti e calcolando l'orario previsto di arrivo). Il pulsante prominente "Avvia" è un invito all'azione chiaro che prepara la transizione di stato successiva.



6.3. Scenario Operativo: Navigazione Attiva, Chevron e Telecamera Dinamica

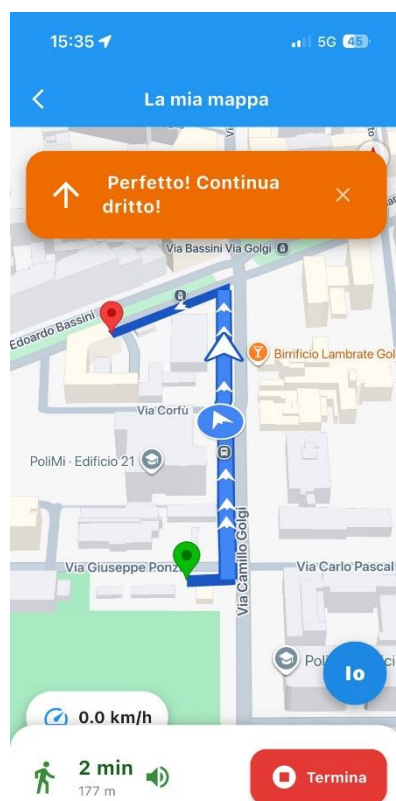
Una volta azionato il comando di partenza, l'interfaccia subisce una radicale metamorfosi atta ad accomodare le esigenze della guida sicura. Lo screenshot ritrae la complessità tecnica celata da una vista pulita: la telecamera non è più fissa in modalità zenitale (top-down), ma applica un "Tilt" (inclinazione prospettica 3D) per consentire di scorgere l'orizzonte e aumenta drasticamente lo zoom.

Visivamente, la polyline del percorso è ora arricchita dalla sovrapposizione dinamica dei marker direzionali (Chevron) bianchi, distanziati a intervalli calcolati regolarmente sulla geometria curva. In alto si staglia l'indicatore delle manovre ("Turn-by-turn banner"), un pannello massiccio e ad alto contrasto che istruisce tempestivamente l'autista a svoltare, indicando in modo netto sia la direzione (freccia iconica) sia la distanza numerica continuamente calata dallo stream del GPS nel 'NavigationMonitor'.



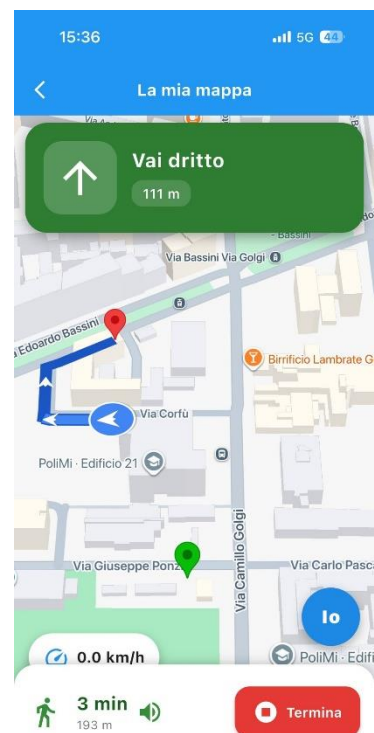
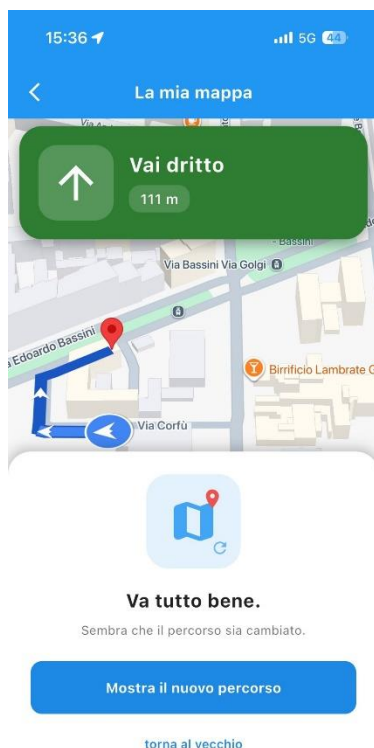
6.4. Apparizione overlay di aiuto

Questa schermata illustra il comportamento contestuale dell'interfaccia utente in prossimità di uno snodo non previsto. L'applicazione implementa un sistema di 'Overlay all'incrocio' che si attiva dinamicamente in base alla distanza dell'utente dal prossimo punto di svolta e alla velocità attuale. L'elemento in sovrapposizione è progettato per catturare l'attenzione in modo non invasivo, presentando informazioni essenziali ad alta leggibilità: un'icona direzionale inequivocabile, l'indicazione testuale dell'azione da compiere e la metratura rimanente all'incrocio. Questa soluzione UX garantisce che le istruzioni di navigazione critiche vengano fornite con il giusto tempismo, mantenendo alto il livello di attenzione dell'utente e riducendo al minimo l'esitazione nei punti decisionali del tracciato.



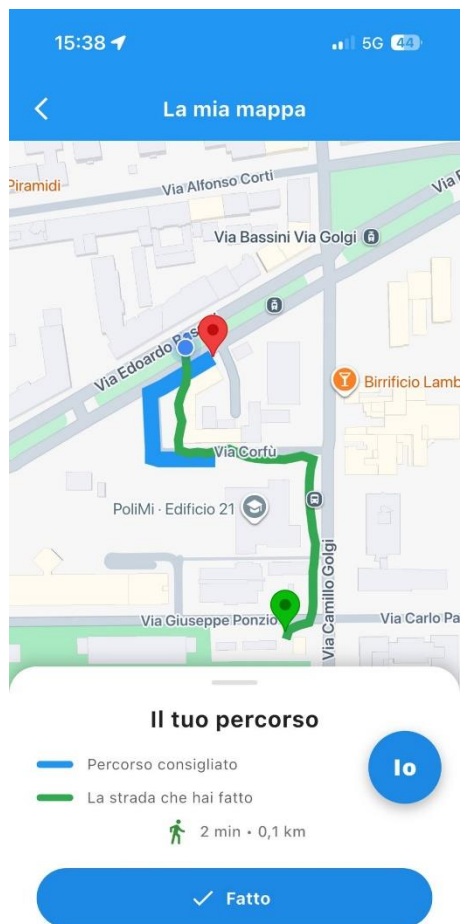
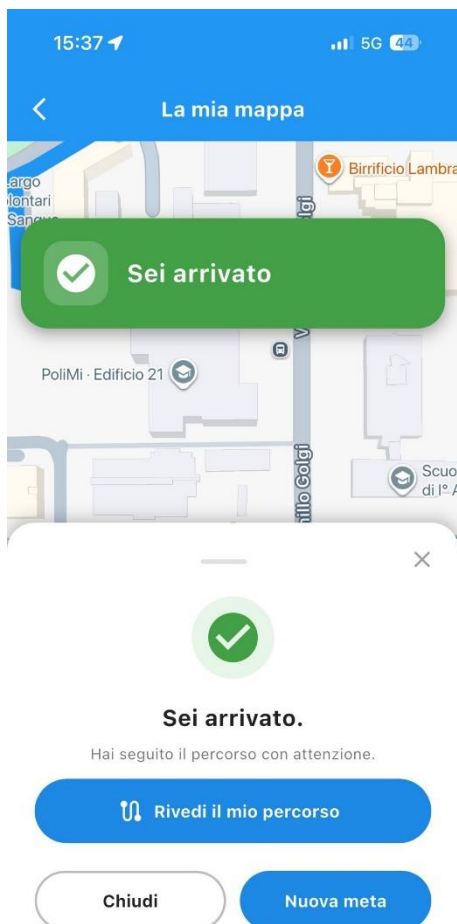
6.5. Ricalcolo percorso

In questo screenshot viene evidenziata la resilienza e la reattività del sistema di navigazione di fronte a deviazioni dal tracciato originario. Quando il servizio di monitoraggio della posizione (GPS) rileva un allontanamento oltre una certa soglia di tolleranza rispetto alla rotta stabilita, l'applicazione innesca istantaneamente e in background la procedura di 'Ricalcolo del percorso'. L'interfaccia informa l'utente dell'operazione in corso tramite un apposito feedback visivo. Non appena i servizi di routing restituiscono i nuovi dati, la polyline sulla mappa viene ridisegnata in tempo reale per mostrare la nuova traiettoria ottimale verso la destinazione. Questo processo avviene in modo completamente trasparente, assicurando un'esperienza di navigazione fluida e senza interruzioni manuali.



6.6. Arrivo a destinazione e schermata di review del percorso

L'immagine rappresenta l'evento conclusivo del flusso di navigazione e la transizione verso la schermata di riepilogo. Non appena le coordinate dell'utente coincidono con l'area di tolleranza della destinazione finale, il sistema interrompe il tracking attivo e notifica il successo tramite un chiaro messaggio visivo di 'Arrivo a destinazione'. Contestualmente, l'applicazione esegue un riposizionamento automatico della telecamera della mappa (zoom-out e re-framing) per inquadrare nella sua interezza il tragitto appena completato. Questa visualizzazione panoramica fornisce all'utente un quadro d'insieme del percorso effettuato, fungendo da schermata di chiusura della sessione prima del ritorno al menu principale.



7. Conclusioni e Sviluppi Futuri

La progettazione e l'ingegnerizzazione di "Smart Navigation App" rappresentano un traguardo notevole nella sintesi e applicazione di algoritmi complessi in un ambiente operativo critico e mobile come quello della navigazione assistita. Questo documento ha esaminato il percorso che ha condotto dall'ideazione teorica all'implementazione concreta, passando attraverso la modellazione formale dei domini e la risoluzione empirica di sfide tecniche considerevoli.

I risultati ottenuti, validati dai test visivi e funzionali, certificano la bontà delle soluzioni architetturali adottate. L'utilizzo di Flutter, unito all'eccellente capacità asincrona di Dart, ha consentito la costruzione di un layer visivo estremamente reattivo (60 FPS su hardware comune). La decisione drastica e pionieristica di distaccarsi dalle implementazioni standard a geofence circolari in favore di logiche analitiche di calcolo delle deviazioni geometriche (point-to-line tracking) e di estrapolazione spaziale delle deviazioni in tempo reale ha conferito al software un'accuratezza e una velocità di ricalcolo competitive o addirittura superiori in specifici contesti urbani complessi rispetto ad alternative blasonate.

In aggiunta, l'aver ingegnerizzato funzionalità di intelligenza contestuale come la "Lateral Road Detection", subordinandola a complessi algoritmi di cooldown e state machine logiche per tutelare l'incolumità visiva del conducente, inquadra il prodotto non come una generica utility basata su mappe, bensì come un software avanzato orientato alla Human-Computer Interaction sicura.

Limitazioni Attuali e Sviluppi Futuri Strategici

Nessun artefatto software sfugge all'opportunità di estensione e perfezionamento. L'attuale build prototipale dell'applicazione, nonostante la robustezza, dipende esclusivamente dalla continuità delle comunicazioni di rete per la risoluzione crittografica dei percorsi e l'estrapolazione delle direttive testuali da Google API. Una potenziale disconnessione nel mezzo dell'erogazione di un itinerario in ambienti remoti comporta attualmente una degradazione del servizio. Pertanto, la priorità assoluta nei successivi cicli di iterazione riguarderà l'ingegnerizzazione di un motore di routing offline e locale (tramite algoritmi su grafi OpenStreetMap preventivamente scaricati). Tali implementazioni coronando la missione di fornire uno strumento perfetto e inscindibile per la navigazione.

8. Allegati e Sitografia

- Manuale Utente Operativo in formato PDF allegato al pacchetto software.
- Google Maps Directions API Documentation:
<https://developers.google.com/maps/documentation/directions>
- Flutter Framework Documentation & Guidelines: <https://flutter.dev/docs>